

Roller Rink 3D

Technical documentation and user manual

Claudia Cornacchia

Contents

1	Project overview	2
2	Development environment and architecture	3
2.1	Core technologies and libraries	3
2.2	Assets	3
2.3	Architecture	4
3	Technical aspects	4
3.1	Hierarchical avatar modelling and animations	4
3.2	Environment and texturing	7
3.3	Lighting and day/night cycle	7
3.4	Physics engine and game logic	8
3.5	Autonomous NPCs and crowd procedural animation	8
3.6	Dynamic camera system	9
4	User Manual	9
4.1	UI and game modes	9
4.2	Controls	10
4.3	Environment and camera controls	10
4.4	Performance settings	10

1 Project overview

Roller Rink 3D is an interactive roller skating simulation set inside an indoor sports arena. The project is on one hand a simulator where the character can perform three different figure skating jumps (Waltz, Flip, and Double Axel) and a spin, while on the other hand, it is a fully playable game.

To satisfy both the simulation and the gaming aspects, the application features three distinct modes: a free-roaming **training mode** where the player can practice movements, jumps, and spins while manually dodging other clone skaters navigating the rink, an **exhibition mode** where the skater follows an elliptical track, so the player can just focus on timing tricks and spins supported by a cheering crowd, and a competitive **challenge mode** with time limits, dynamic obstacles, and score tracking.

Key features of the project include:

- **Procedural hierarchical avatar:** a skater built from scratch using simple geometries, animated via real-time forward kinematics and keyframe interpolation without any pre imported animations.
- **Procedural architecture:** the entire indoor arena, including the elliptical rink with dynamic doors, the stadium seats and the roof, is mathematically generated via parametric geometry.
- **Custom physics and collisions:** a lightweight custom physics engine managing skater velocity, railing collisions, NPC avoidance, fall mechanics and wheel rolling.
- **Texturing:** physically based rendering applied to the wooden rink, together with procedurally generated canvas textures for cloth banners and wheels.
- **Dynamic lighting:** a day/night cycle that automatically triggers indoor LED and adjusts the global illumination, complete with shadow mapping.
- **Dynamic camera system:** a multi-state camera (Orbit, Follow, Stands) with smooth interpolation.

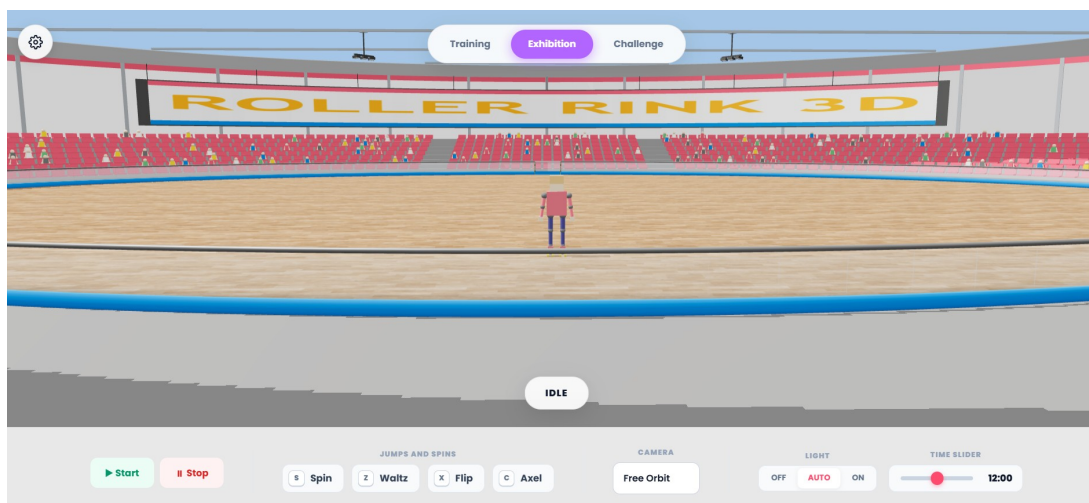


Figure 1: Overview of the roller rink

2 Development environment and architecture

2.1 Core technologies and libraries

Operating entirely within the browser, the application is driven by an `index.html` entry point that bootstraps an ES6 module architecture, ensuring separation between game logic and rendering. For the 3D graphics, the project relies on the **Three.js** library, which provides a robust abstraction over standard **WebGL**, facilitating management of the hardware-accelerated 3D rendering, materials and lighting equations. To manage user-driven camera navigation, the official Three.js add-on module **OrbitControls** was integrated. Finally, the project integrates the external **Tween.js** library, which handles keyframe interpolations and non-linear easing functions.

2.2 Assets

- **Wood textures:** A set of raster images sourced from AmbientCG are applied to the wooden rink to define a true BRDF material model. By using Roughness and Normal maps to simulate the **microscopic structures** of the wood, the material achieves realistic reflections.



Figure 2: Diffuse, Roughness, and Normal maps applied to the wooden rink

- **Raster images:** I used a few PNG files for the roller skater silhouettes, sourced from Flaticon. Since they are in RGBA format, they include an alpha channel for opacity. This allowed me to dynamically combine them with the procedural colored backgrounds of the banners using alpha blending. I also used a raster image for the skater's face (Minecraft style), which I inserted directly onto a colored Canvas 2D applied to the front of the head box.

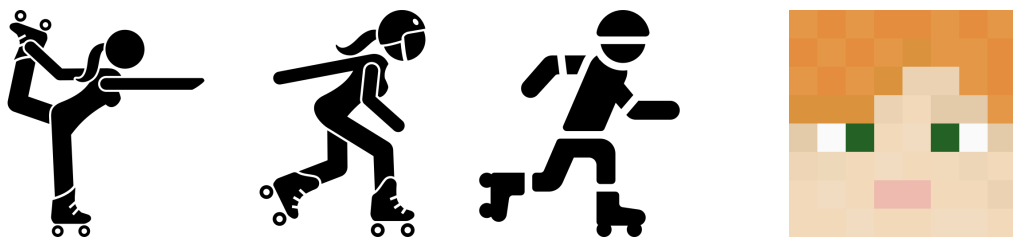


Figure 3

2.3 Architecture

I organized the codebase into different modules to cleanly separate the game logic from the 3D rendering:

- **objects/**: This directory handles the instantiation and spatial configuration of all 3D geometries and hierarchical models. Key modules include:
 - **skater.js**: builds the hierarchical skater and applies local pivot translations.
 - **architecture.js**, **rink.js**, **seating.js**, **doors.js** : generate the procedural arena geometries.
 - **clones.js** and **spectators.js**: manage the obstacle clones via an Object Pool, and the animated NPC crowd using instancing.
 - **lighting.js**: handles the Hemisphere and Directional lights, shadow mapping, and the day/night cycle.
- **logic/**: The simulation core calculating the kinematics, spatial coordinates, and physical vectors before they are processed by the renderer:
 - **physics.js**: custom engine calculating skater momentum, friction, and boundary collisions.
 - **animations.js**: manages all the skater animations.
 - **camera.js**: computes the spatial coordinates and look-at vectors for the dynamic transitions between Orbit, Follow, and Stands modes.
 - **input.js**: captures user keystrokes.
- The root files tie the system together. **state.js** holds all the global variables. **renderer.js** configures the WebGL rendering pipeline, tone mapping, and pixel ratio. **main.js** manages the continuous game loop, updating the math logic and drawing the 3D scene at every frame. Finally, **hud.js** controls the HTML interface (tab, slider, scores), keeping the UI logic separated from the 3D rendering loop.

3 Technical aspects

3.1 Hierarchical avatar modelling and animations

The skater avatar was developed entirely via code using a custom tree-structured model. Figure 4 illustrates the different body parts modelled using basic primitives and their spatial arrangement.

The skater’s anatomy is built upon a central Torso mesh, which acts as the root parent for all the other limbs. By default, Three.js places the origin of a mesh at its geometric center, so rotating it directly would cause limbs to spin around their midpoints rather than the anatomical joints. To solve this, I used invisible **THREE.Group** nodes as the actual rotation pivots. Then, I applied a vertex translation directly to the limb’s geometry ($T_y = -\frac{\text{length}}{2}$), shifting the vertices downwards, so that the local coordinate origin is relocated from the center of the mesh to its top extremity, aligning with the group’s pivot. Spatial coordinates connecting the nodes are defined entirely in the Local Space of their parent (e.g., the Knee group is translated locally by $Y = -0.40$ relative to

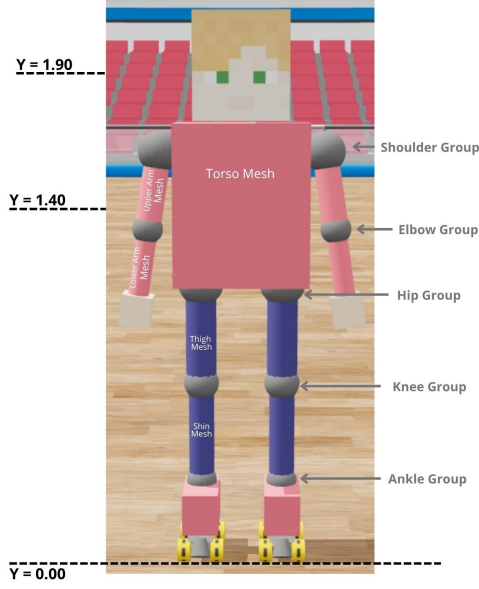


Figure 4: In-game visual breakdown of the skater avatar. The dashed lines highlight the global elevation ($Y = 1.40$) applied to the Torso to align the skates with the ground ($Y = 0.00$). The grey spheres act as visual markers for the anatomical joints.

the Hip). To make this connections visible, I placed dark sphere exactly at the rotation pivots (as seen in Figure 4), this also hides the hard edges of the rotating cylinders. Figure 5 summarizes this mathematical setup and the resulting tree-structured hierarchy.

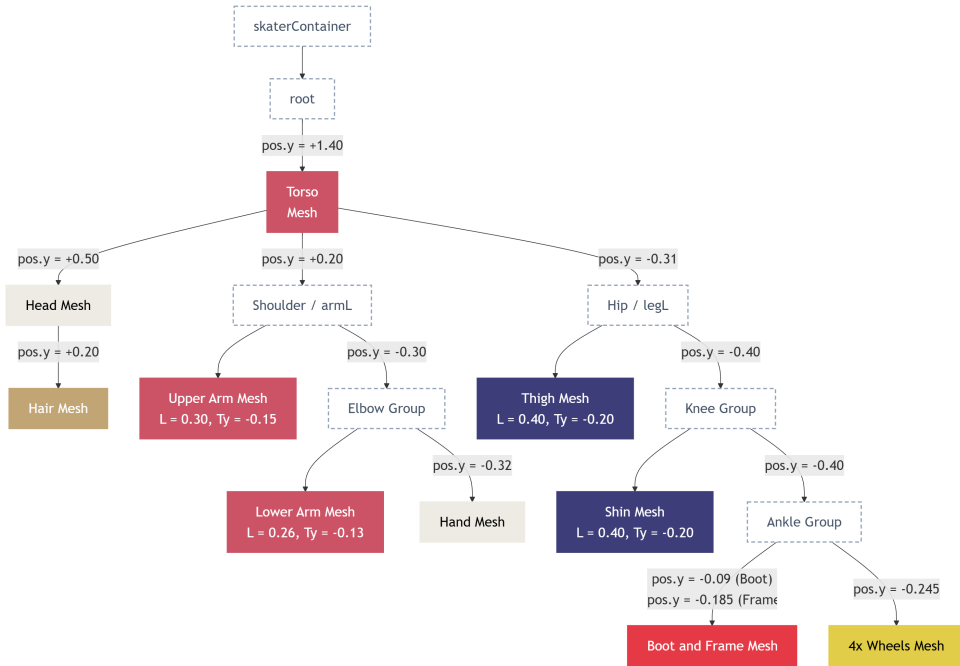


Figure 5: Tree-structured hierarchy of the skater model

This strict parent-child relationship ensures that rotating a parent node (e.g., bending the knee) automatically translates and rotates all its descendants (the ankle, the boot, and the wheels) in the World Space.

Complex multi-stage actions, such as the figure skating jumps (Waltz, Flip, Double Axel) and the fall, are modeled via **Keyframe animation**. I first defined the target local rotations for each key anatomical pose (e.g., preparation, take-off, flight, landing). Then I used the Tween.js library to interpolate these values over time. To avoid robotic snapping and ensure fluid movements, I applied non-linear easing functions which guarantee

C^1 parametric continuity across the animation states. To correctly determine the exact joint angles, I followed the Right-Hand Rule natively used by the Three.js coordinate system. By aligning the right thumb with the positive axis, the curled fingers indicate the positive direction of the rotation (X for Pitch, Y for Yaw, Z for Roll). To speed up the animation workflow, I created the quick reference schema shown in Figure 6. The left image illustrates the Right-Handed coordinate axes; for visual clarity, these axes are depicted only on the root torso, even though every hierarchical node rotation (e.g., shoulders, knees) is computed within its own Local Space. The right image translates this rule into a logical decision tree, summarizing the angle signs (+/-) required to bend or extend each specific limb based on its construction direction.

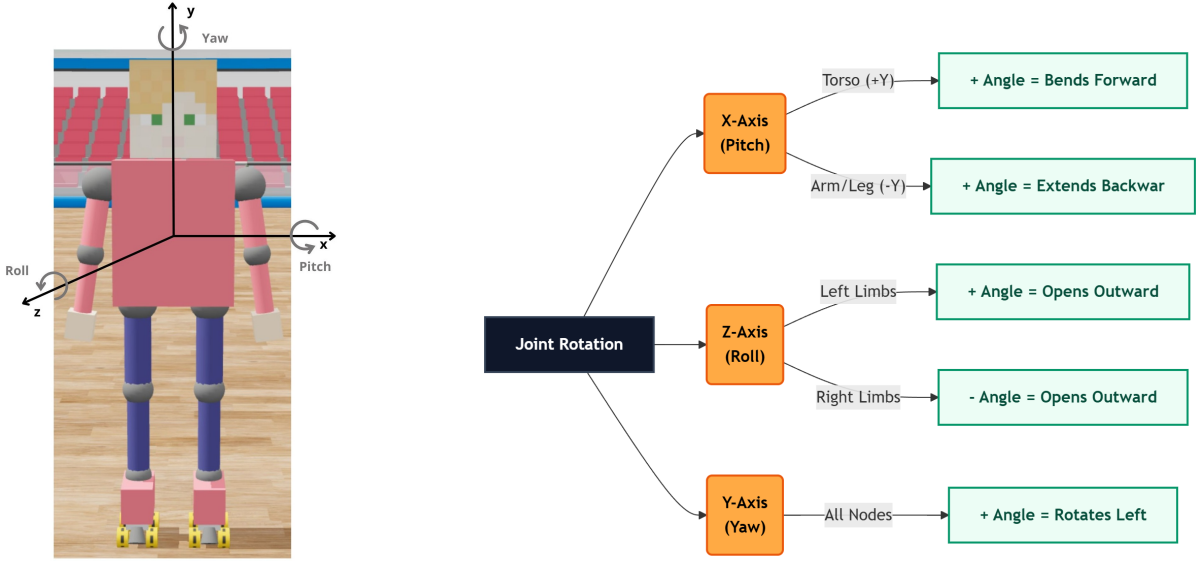


Figure 6: Left: skater's local coordinate axes (depicted on the torso for brevity). Right: angle sign determination schema based on the Right-Hand rule.

The spin animation, is instead a **Procedural animation** computed frame-by-frame. During the entry phase, a Hermite Spline blending function (the smoothstep polynomial $3t^2 - 2t^3$) is used to guarantee C^1 parametric continuity, ensuring the avatar eases smoothly into the pose with zero initial and final velocity. Conversely, the rapid transition to the tight Upright spin uses standard linear interpolation (C^0) to simulate a sudden muscle contraction. This procedural loop simulates the Conservation of Angular Momentum, as the skater pulls their arms inward (decreasing their moment of inertia), the frame-by-frame angular velocity mathematically increases. Finally, to prevent the avatar from stopping at an off-axis orientation, a realignment is applied by combining a modulo 2π operation with an exponential smoothing function (Lerp):

$$R_{new} = R_{current} + (R_{target} - R_{current}) \cdot \alpha \quad (1)$$

(where α is a constant smoothing factor). In this way the rotation is smoothly pulled toward the nearest perfect 360° multiple, keeping the skater aligned with the track.

To eliminate the optical illusion of the skater simply sliding, the wheels' rotation follows the physical principles of pure rolling kinematics ($\Delta\theta = v/r$). I also wrapped a procedural 2D pattern around the wheels to make the wheels' movement visually perceivable.

3.2 Environment and texturing

The indoor arena centers around an **elliptical skating rink**. To simulate the Microfacet BRDF of a varnished parquet, I used three specific raster maps (Figure 2), a Diffuse map defining the base colors, a Normal map computing 3D micro-shadows along the wood grooves and a Roughness map that controls how light bounces and scatters across the surface to shape the specular reflection. I also used Anisotropic filtering to prevent undersampling artifacts. The rink’s perimeter is enclosed by protective boards, and transparent glass panels all procedurally generated by sweeping a TubeGeometry along a single Catmull-Rom spline. To create the glass panels I applied an extreme vertical scaling to stretch the thin tube.

Surrounding the rink, the **grandstands** consist of thousands of identical concrete steps and seats. To prevent CPU-to-GPU bandwidth bottlenecks, I optimized their rendering using Hardware Instancing. Instead of processing each object individually, a single geometric prototype is loaded into the GPU memory just once. Then, an array of 4×4 affine transformation matrices is supplied to dynamically position, rotate, and scale each copy, to allow the graphics card to render the entire stadium seating in a single Draw Call.

Above the track, **hanging banners** decorate the curved perimeter walls, employing two distinct texturing techniques. The two main title banners were generated procedurally at runtime via an off-screen HTML Canvas. The letters and colors are dynamically painted onto a 2D grid via code, acting as a procedural texture function ($c = f(u, v)$). Conversely, to create the team banners suspended from the ceiling I used the skaters raster images (Figure 3) that are equipped with an RGBA alpha channel. To assign distinct parametric background colors to the different images, I used the Alpha Blending equation:

$$c = \alpha_f c_f + (1 - \alpha_f) c_b \quad (2)$$

The environment is enriched with **architectural details**, such as glass and metal doors and a wall clock. To optimize rendering performance, I drew the clock’s dial and its twelve hour marks via the HTML5 Canvas API using trigonometric functions, applying the resulting canvas as a single CanvasTexture. The hour and minute hands are elongated BoxGeometry within hierarchical groups. To guarantee accurate rotations, I translated their vertices along the local Y-axis shifting the rotation pivot from the center to their base. Finally, to synchronize with the dynamic day/night time slider, their Z-axis rotations are updated continuously via modulo operators.

3.3 Lighting and day/night cycle

The global lighting features a dynamic 24-hour day/night cycle. To achieve a seamless transition of the atmospheric sky colors, I interpolated them using a Hermite Spline blending function (the smoothstep polynomial).

Celestial bodies and starfield: the Sun and Moon are represented by a unified spherical geometry that orbits the arena using trigonometric functions. During the day, the Sun acts as a Directional Light (simulating parallel rays). Following the Lambertian (Diffuse) Reflection Model, the ground naturally dims at sunset because the geometric term ($\cos \theta$) between the surface normal and the light direction decreases, spreading incoming light over a larger area. At night, a procedural starfield fades in, generated by randomly sampling spherical coordinates (θ, ϕ) and mapping them into 3D Cartesian coordinates (x, y, z) to form a celestial dome above the arena.

Indoor pendant lights: as the natural light drops, the arena’s automated suspension lights turn on. Each lamp uses a Point Light to radiate energy spherically in all directions. The user can manually override this automated day/night logic via the interface, explicitly forcing the lights on or off at any time.

Shadow mapping: a point is in shadow if the light source cannot see it. To compute this, a virtual second camera placed at the light source generates a Depth Map, storing only the distances of the closest objects. During rendering, each 3D point’s distance to the light is compared against this map: if it is greater than the stored value, the light is blocked by another object and the point is rendered in shadow.

3.4 Physics engine and game logic

A custom, lightweight physics engine calculates the skater’s position by integrating acceleration, velocity, and simulated friction over time. The main player’s behaviour is governed by a **Finite State Machine** (transitioning between Idle, Skating, Jumping, and Falling states). To handle rink boundaries, the system mathematically evaluates the implicit equation of an ellipse ($\frac{x^2}{a^2} + \frac{z^2}{b^2} \leq 1$). If the boundary is breached, an inelastic collision is triggered, applying a kinematic recoil, or a fall if the skater hits the wall during a jump. Dynamic collisions between the player and the environment are evaluated using a Bounding Sphere intersection check ($D = \sqrt{(x_2 - x_1)^2 + (z_2 - z_1)^2}$). In Challenge Mode, impacting a clone skater forcefully interrupts the player’s animation state machine, triggering the fall animation and deducting score points.

3.5 Autonomous NPCs and crowd procedural animation

The arena is also populated by **autonomous clone skaters** and **audience on the stands**.

The **clone skaters** are managed via an Object Pool pattern, meaning that all required geometries are pre-allocated at startup, dynamically toggling their visibility to prevent frame drops caused by real-time memory allocation. Driven by their own **Finite State Machine**, they navigate the rink and avoid each other using a repulsive Separation Force vector. Based on a probabilistic function, they can autonomously decide to take a break, they transition to an exiting state, trigger the physical opening of the dynamic rink gates, navigate to the bleachers, and interpolate their hierarchical joints into a sitting posture.

Similarly, the **seated audience** was implemented using a procedural approach using only six THREE.InstancedMesh objects (representing the torso, head, and four limb segments) pre-allocated at startup. This guarantees that rendering hundreds of spectators requires strictly 6 Draw Calls. The crowd’s movement rely on mathematical functions evaluated per-frame on the CPU to dynamically update the 4×4 transformation matrices of each instance. When the player lands a jump or performs a spin, a global event triggers a Cheer state. Visually, the spectators bounce enthusiastically, mathematically, this is achieved by modifying their height using high-frequency absolute sine waves ($|\sin(t \cdot f)|$) multiplied by a linear decay function. The amplitude and duration of this bounce adapt to the difficulty of the landed trick (an Axel produces a higher and longer cheer). To prevent the unnatural visual artifact of the entire crowd moving in perfect unison, a randomized phase delay is injected into the time variable (t) of each instance, resulting in an organic dispersion of the movement. Furthermore, the spectator’s arms are animated by blending two target directions, a resting direction (pointing down) and a cheering

direction (pointing up). Depending on the spectator's excitement level, it is computed the intermediate pointing direction. This final vector is then automatically converted into a 3D rotation and applied to the arm.

3.6 Dynamic camera system

A multi-state camera system dynamically updates the viewing transformations to map World Space into Camera Space:

- **Orbit mode:** uses THREE.OrbitControls module to allow free exploration of the environment via mouse dragging (panning) and scrolling (zooming).
- **Follow mode:** applies Linear Interpolation (Lerp) to smoothly track the skater's position over time, while dynamically updating the view direction to keep the avatar in focus.
- **Stands mode:** anchors the camera origin to a fixed World Space position (the bleachers) while continuously recomputing the view direction vector to track the player.

To prevent the camera from clipping through the arena walls, the Frustum coordinates are constrained using the same elliptical equation applied in the physics engine.

4 User Manual

4.1 UI and game modes

Upon launching the application, the user is presented with an HTML/CSS overlay sitting on top of the WebGL canvas. The top navigation bar allows the player to switch between the three core game modes:

- **Training Mode (Default):** the player can freely skate around the arena using the arrow keys. It is ideal for testing the physics, learning the timing of the jumps, and practicing how to dodge the NPC skaters.
- **Exhibition Mode:** the skater follows a perfect elliptical path around the rink. The player only needs to focus on executing acrobatics at the right moment. Pressing **Enter** toggles between skating and pausing the exhibition.
- **Challenge Mode:** a competitive mode. The user selects a difficulty (Easy, Medium, or Hard), which determines the time limit (from 2:00 down to 1:00 minute), the number of active NPC skaters on the rink (from 2 up to 5), and the target score. The player must chain jumps and spins to accumulate points while dodging rival skaters (-150 points penalty) and arena walls (-50 points penalty). Collecting bonus stars thrown by the crowd grants +150 points, while hitting malicious cones deducts -100 points and breaks the skater's momentum.

The crowd is only visible in challenge and exhibition mode. The bottom HUD dynamically updates to display the current movement state or the action being performed (e.g., "Double Axel!!"), or the skater's speed percentage.

4.2 Controls

To accommodate different play styles, the application features a dual control scheme. The user can perform actions using the physical keyboard (highly recommended for Challenge mode), or by clicking the dedicated buttons located on the bottom HUD (ideal for the mouse-only experience during Exhibition mode). The controls are intentionally locked during mid-air flight and fall animations to respect the physical commitment of the jumps.

Keyboard Key / UI Button	Action
Arrow Up	Skate forward / Push (Hold to accelerate)
Arrow Down	Brake / Decelerate
Arrow Left / Right	Steer left / right
Z / Waltz Btn	Perform a Waltz Jump (100 pts)
X / Flip Btn	Perform a Flip Jump (250 pts)
C / Axel Btn	Perform an Axel Jump (500 pts)
S / Spin Btn	Perform a Fast Spin / Pirouette (300 pts)
M	Switch Game Mode / Abort active Challenge
Enter	Start Challenge / Play/Pause Exhibition

4.3 Environment and camera controls

Alongside the trick buttons, the bottom HUD contains a control panel that allows the user to manipulate the scene in real-time:

- **Camera mode:** a dropdown menu to switch the perspective between Free Orbit (manual rotation), Follow (dynamic third-person tracking), and Stands (spectator's viewpoint from the bleachers).
- **Light controls:** three dedicated buttons (Auto / On / Off) allow the player to override the automated indoor LED projectors.
- **Time of day slider:** allows the user to manually control the sun and moon sky position, changing the atmosphere from sunrise to deep night.

4.4 Performance settings

To ensure smooth execution across different hardware, a settings panel is accessible via the gear icon located in the top-left corner. **Troubleshooting tip:** if the game experiences a frame rate (FPS) drop, users are advised to adjust the following settings:

- **Graphics quality selector:** dynamically scales down the WebGL rendering resolution. This drastically reduces the number of pixels processed by the Fragment Shader, preventing GPU bottlenecks on high-density displays.
- **Shadows toggle:** completely disables the Shadow Map calculation. Skipping the secondary render pass needed to generate the Depth Map effectively halves the geometry workload, providing a performance boost for devices without a dedicated GPU.